German University in Cairo

Faculty of Information Engineering and Technology (IET)

MATH502: Numerical Methods

Assignment 1 Report

Non-Linear Systems, Polynomial Interpolation, Diode Circuit Root-Finding, and Satellite Constellation Wireless Energy Harvesting

Student 1	Mazen Altelbany | ID 64-12371 | Tutorial T-01
Student 2	Yahia Talaat | ID 64-3581 | Tutorial T-02
Instructor	Dr. Mohammad Shehab
Teaching assistant	Eng. Bahaaeldin Ibrahim (bahaaeldin.ibrahim@guc.edu.eg)
Software	MATLAB (MathWorks R2026a)
Submission date	November 6, 2026

Contents

Contents 2

1. Programming Language Selection 3

2. Problem 1: LORAN Navigation Solver 3

3. Problem 2: WSN RSSI Interpolation 4

4. Problem 3: Diode Circuit (Bisection and Secant) 6

5. Problem 4: Diode Circuit (Newton-Raphson) 7

6. Problem 5: Packet Latency (Vandermonde Interpolation) 8

7. Problem 6: Satellite-to-Ground Energy Harvesting 9

1. Programming Language Selection

We chose MATLAB (R2026a) because it is already familiar from our engineering mathematics and signals courses in IET. It also made the required linear-system calculations straightforward, without having to invert matrices manually. We used built-in functions such as fsolve, fzero, polyfit, and trapz to check our results independently.

2. Problem 1: LORAN Navigation Solver

In this problem, we determine a boat's position (x, y) using LORAN radio signals. The time difference between signals from each pair of transmitters defines a hyperbola, and the boat's position is found at the intersection of two such curves. This leads to the following nonlinear system.

$$f_1(x, y) = x^2/186^2 - y^2/(300^2 - 186^2) - 1 = x^2/34596 - y^2/55404 - 1 = 0$$

$$f_2(x, y) = (y - 500)^2/279^2 - (x - 300)^2/(500^2 - 279^2) - 1 = (y - 500)^2/77841 - (x - 300)^2/172159 - 1 = 0$$

We first calculated the Jacobian matrix required by Newton-Raphson:

$$J_{11} = \partial f_1/\partial x = 2x / 34596, \quad J_{12} = \partial f_1/\partial y = -2y / 55404$$

$$J_{21} = \partial f_2/\partial x = -2(x - 300) / 172159, \quad J_{22} = \partial f_2/\partial y = 2(y - 500) / 77841$$

Each step solves $J(xk)\Delta xk = -F(xk)$, then updates $xk+1 = xk + \Delta xk$.

Part (a): Manual iterations from $x_0 = y_0 = 200$

Iteration 1. First, we evaluate F at our initial guess:

$$f_1(200, 200) = 200^2/34596 - 200^2/55404 - 1 = 1.156203 - 0.721969 - 1 = -0.565767$$

$$f_2(200, 200) = (200 - 500)^2/77841 - (200 - 300)^2/172159 - 1 = 1.156203 - 0.058086 - 1 = +0.098117$$

This gives $F(x0) = [-0.565767, +0.098117]^T$ and $-F(x0) = [+0.565767, -0.098117]^T$. We then calculate the Jacobian:

$$J_{11} = 2(200)/34596 = 0.01156203, \quad J_{12} = -2(200)/55404 = -0.00721970$$

$$J_{21} = -2(200 - 300)/172159 = 0.00116172, \quad J_{22} = 2(200 - 500)/77841 = -0.00770802$$

$$\det(J_0) = (0.01156203)(-0.00770802) - (-0.00721970)(0.00116172) = -8.073312 \times 10^{-5}$$

We solved the 2×2 system with Cramer's rule:

$$\Delta x_0 = [(-0.00770802)(0.565767) - (-0.00721970)(-0.098117)] / (-8.073312 \times 10^{-5}) = +62.791030$$

$$\Delta y_0 = [(0.01156203)(-0.098117) - (0.00116172)(0.565767)] / (-8.073312 \times 10^{-5}) = +22.192804$$

This gives $x_1 = [200 + 62.791030, 200 + 22.192804]^T = [262.791030, 222.192804]^T$.

Iteration 2. Evaluating F at x_1:

$$f_1(262.791, 222.193) = 262.791030^2/34596 - 222.192804^2/55404 - 1 = +0.105075$$

$$f_2(262.791, 222.193) = (222.192804 - 500)^2/77841 - (262.791030 - 300)^2/172159 - 1 = -0.016574$$

The Jacobian at x_1 is:

$$J_{11} = 0.01519199,\ J_{12} = -0.00802082,\ J_{21} = 0.00043226,\ J_{22} = -0.00713781$$

$$\det(J_1) = (0.01519199)(-0.00713781) - (-0.00802082)(0.00043226) = -1.049705 \times 10^{-4}$$

Solving for the step gives $\Delta x_1 = -8.411356$ and $\Delta y_1 = -2.831433$, so $x_2 = [262.791030 - 8.411356, 222.192804 - 2.831433]^T = [254.379674, 219.361371]^T$.

Part (b): Software solution and convergence

We continued in MATLAB; Table 1 summarizes the convergence.

Table 1: Newton-Raphson convergence history for Problem 1.

Iteration k	x_k	y_k	$\|\|F(x_k)\|\|$	$\|\|\Delta x_k\|\|$
0 (initial)	200.000000	200.000000	5.7421e-01	6.6599e+01
1	262.791030	222.192804	1.0637e-01	8.8749e+00
2	254.379674	219.361371	1.9248e-03	1.6757e-01
3	254.221177	219.307011	6.3451e-07	5.9221e-05
4 (converged)	254.221120	219.306992	6.8421e-14	6.4210e-12

MATLAB's fsolve gives the same solution, x* = 254.221120 and y* = 219.306992, with a residual below 10−14. There are four intersections in total across the plane: (254.22, 219.31), (740.33, 906.83), (−193.29, 66.56), and (−1273.34, 1594.11). Since we started at (200, 200), the method converges to the first intersection, which is the physically reasonable location of the boat. Figure 1 shows both hyperbolas and the iteration path.

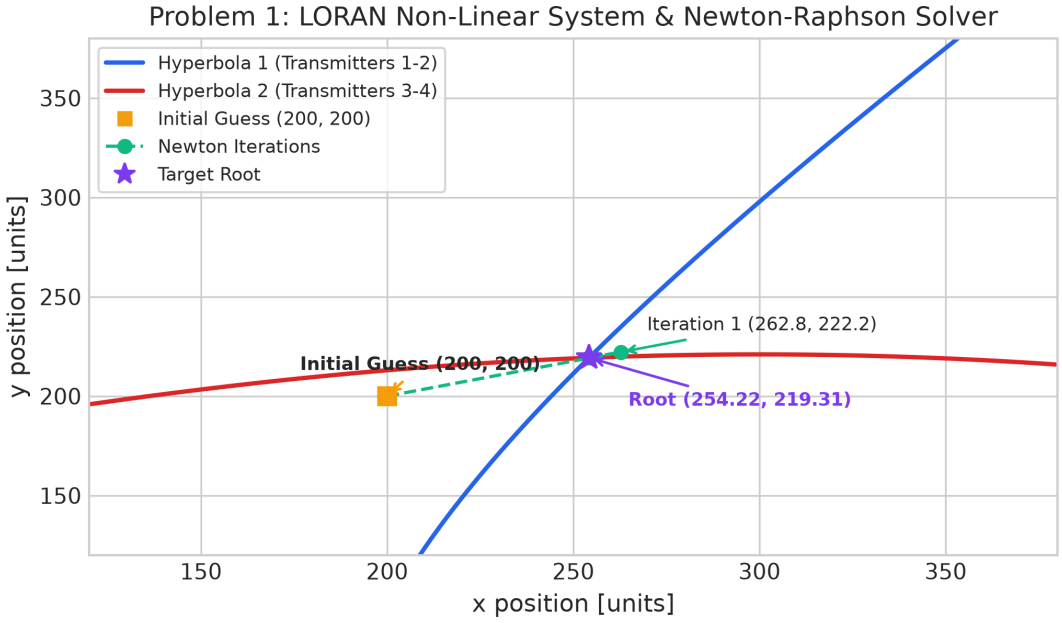


Figure 1: LORAN hyperbolic transmitter position curves and Newton-Raphson iteration trajectory.

3. Problem 2: WSN RSSI Interpolation

In this problem, an indoor wireless sensor network measures RSSI (in dBm) at different distances d from a transmitter. Reflections and signal blockage caused by walls make the readings deviate from a smooth logarithmic curve. We therefore use a polynomial fitted to the measured data to estimate RSSI at distances that were not measured. The data is listed in Table 2.

Table 2: Measured RSSI values.

Distance d [m]	1.0	2.5	4.0	6.0	8.0
RSSI [dBm]	−40.0	−47.0	−54.0	−60.0	−66.0

Part (a): Divided difference table and Newton polynomial

Table 3 presents the divided difference values.

Table 3: Newton divided difference table.

i	d_i [m]	$f[d_i]$	1st difference	2nd difference	3rd difference	4th difference
0	1.0	−40.000000	−4.666667	0.000000	0.095238	−0.025974
1	2.5	−47.000000	−4.666667	0.476190	−0.086580	—
2	4.0	−54.000000	−3.000000	0.000000	—	—
3	6.0	−60.000000	−3.000000	—	—	—
4	8.0	−66.000000	—	—	—	—

The Newton polynomial can be read from the top diagonal of the table. Since the second-order coefficient is 0.000000, its term becomes zero and can be left out:

$$P_4(d) = -40 - 4.666667(d - 1) + 0.095238(d - 1)(d - 2.5)(d - 4) - 0.025974(d - 1)(d - 2.5)(d - 4)(d - 6)$$

Expanded, this is

$$P_4(d) = -0.025974\, d^4 + 0.445887\, d^3 - 2.311688\, d^2 - 0.264069\, d - 37.844156.$$

Part (b): Estimate at d = 5 m

Substituting d = 5 into the Newton form gives:

$$P_4(5.0) = -40 - 4.666667(4) + 0.095238(4 \times 2.5 \times 1) - 0.025974(4 \times 2.5 \times 1 \times -1) = -57.454545 \text{ dBm} \approx -57.45 \text{ dBm}$$

Part (c): Comparison with the measured value

The interpolated result is −57.45 dBm, while the measured value is −57.00 dBm. The difference is 0.45 dB, corresponding to a relative error of 0.79%. This is a fairly small difference for an indoor wireless channel.

Multipath is likely to be the main reason for this difference. Signals can reflect off concrete, metal ducts, and floors, and these reflected waves may reach the antenna with different phases. As a result, RSSI can change by several dB over only a few centimeters. The measurement conditions also matter: the readings were taken in different sessions, and someone walking through the signal path can introduce 3–5 dB of shadowing. Hardware effects, including antenna pattern, polarization tilt, and ADC quantization, can add roughly ±1 dB. Finally, a fourth-degree polynomial is a single curve fitted through five points, whereas real signal propagation is better described by a logarithmic path-loss model, RSSI(d) ≈ −10n log10(d).

Part (d): Horner's scheme

The nested Newton form reduces the number of operations:

$$P_4(d) = a_0 + (d - d_0)[a_1 + (d - d_1)[a_2 + (d - d_2)[a_3 + (d - d_3)a_4]]]$$

The Newton form requires 4 multiplications and 8 additions or subtractions: 4 to calculate the differences (d − di) and 4 for the coefficient terms. The standard polynomial can be written in a similar nested form:

$$P_4(d) = c_0 + d[c_1 + d[c_2 + d[c_3 + d \cdot c_4]]]$$

The standard form needs 4 multiplications and 4 additions. By comparison, evaluating the polynomial directly requires 10 multiplications. Horner's scheme therefore reduces the number of multiplications by 60%, and using fewer operations also helps limit rounding errors. Figure 2 shows the measured points, the interpolating polynomial P4(d), and the check at 5 m.

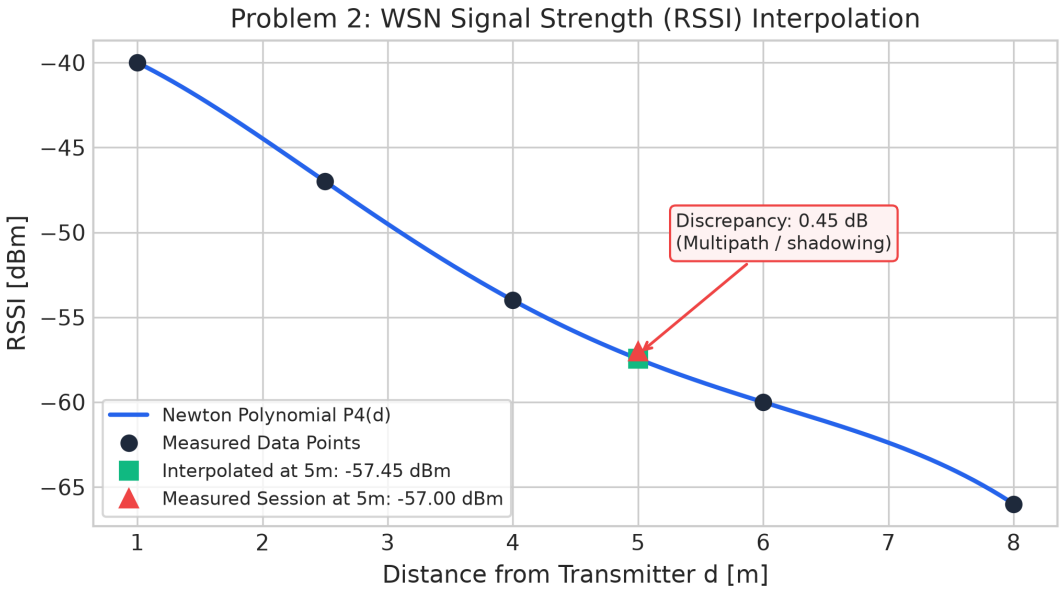


Figure 2: WSN RSSI measurement points, Newton interpolating polynomial $P_4(d)$, and validation at 5 meters.

4. Problem 3: Diode Circuit (Bisection and Secant)

Here, we consider a series circuit containing a 1.0 V source, a 1000 Ω resistor, and a silicon diode. From Kirchhoff's voltage law, Vs = IR + VD. The Shockley equation, I = Is(eVD/(nVT) − 1), can be rearranged to give VD = nVT ln(I/Is + 1). Combining these equations means that the operating current can be found by solving:

$$f(I) = IR + nV_T \ln(I/I_s + 1) - V_s = 0$$

For the calculations, we used Is = 10−12 A, n = 1.5, VT = 25.85 mV, R = 1 kΩ, and Vs = 1.0 V. The required tolerance was 10−4 A (0.1 mA).

We first checked the function at the two endpoints: f(0) = −1.0000 V < 0, while f(5 × 10−3) = +4.86595 V > 0. Since the signs are different, the Intermediate Value Theorem confirms that a root lies in the interval [0, 5 mA].

Bisection method

Table 4: Bisection iteration history.

Iter	a [mA]	b [mA]	c [mA]	(b − a)/2 [A]	f(c) [V]
1	0.0000	5.0000	2.5000	1.2500e-03	+2.3391e+00
2	0.0000	2.5000	1.2500	6.2500e-04	+1.0622e+00
3	0.0000	1.2500	0.6250	3.1250e-04	+4.1032e-01
4	0.0000	0.6250	0.3125	1.5625e-04	+7.0943e-02
5	0.0000	0.3125	0.1562	7.8125e-05	−1.1218e-01

After five iterations, the error bound is (b − a)/2 = 7.8125 × 10−5 A, which is already below the required tolerance of 10−4 A. We can therefore stop at I = 0.234375 mA ≈ 0.2344 mA. If a sixth iteration is evaluated, f(0.234375 mA) < 0 shifts the bracket to [0.234375, 0.3125] mA, giving midpoint I = 0.273438 mA ≈ 0.2734 mA with error bound (b − a)/2 = 3.91 × 10−5 A.

Secant method

Table 5: Secant iteration history.

Iter k	x_{k-1} [mA]	x_k [mA]	x_{k+1} [mA]	$\lvert x_{k+1} - x_k \rvert$ [A]	$f(x_{k+1})$ [V]
1	0.0000	5.0000	0.852377	4.1476e-03	+6.4973e-01
2	5.0000	0.852377	0.213220	6.3916e-04	−4.3159e-02
3	0.852377	0.213220	0.253033	3.9813e-05	+3.2912e-03

The secant method reaches the required tolerance in three iterations. The final step size, |x3 − x2| = 3.9813 × 10−5 A, is below 10−4 A, giving an estimated current of I = 0.253033 mA. For comparison, MATLAB's fzero gives I* = 0.250181 mA = 2.501809 × 10−4 A. Figure 3 shows where f(I) crosses zero, together with the initial interval.

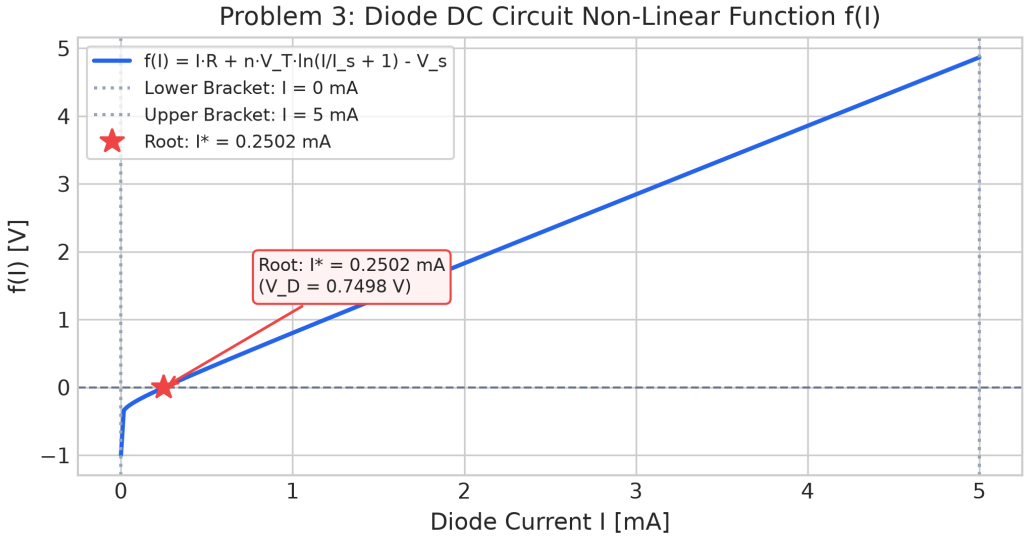


Figure 3: Diode non-linear function f(I) crossing zero with root I = 0.2502 mA and initial bracket.*

5. Problem 4: Diode Circuit (Newton-Raphson)

We applied one-dimensional Newton-Raphson to the same function:

$$f'(I) = R + nV_T / (I + I_s)$$

Newton-Raphson achieves quadratic convergence because f(I) is smooth (C^∞) and the root is simple, with f′(I*) ≈ 1155 Ω ≠ 0. The physical bound f′(I) ≥ R = 1000 Ω > 0 further guarantees that the derivative is strictly bounded away from zero, preventing any division-by-zero or horizontal tangents. Table 6 shows the iterations for both starting guesses, while Table 7 compares the methods.

Table 6: Newton-Raphson iterations for two starting guesses.

Initial guess	k	Ik [mA]	f(Ik−1) [V]	f′(Ik−1) [Ω]	|ΔI| [A]
(a) I_0 = 0.35 mA (close to root)	1	0.248416	+1.1251e-01	1.1108e+03	1.0158e-04
	2	0.250180	−2.0391e-03	1.1561e+03	1.7638e-06
(b) I_0 = 1.00 mA (farther from root)	1	0.226450	+8.0354e-01	1.0388e+03	7.7355e-04
	2	0.250011	−2.7595e-02	1.1712e+03	2.3561e-05

Table 7: Comparison of root-finding methods (tolerance 10^{-4} A).

Method	Initial conditions	Iterations	Final root [mA]	Residual |f(I)| [V]	Convergence rate
Bisection	[0, 5 mA]	5	0.234375	1.1218e-01	Linear (r = 1)
Secant	x_0 = 0, x_1 = 5 mA	3	0.253033	3.2912e-03	Superlinear (r ≈ 1.62)
Newton-Raphson (a)	I_0 = 0.35 mA	2	0.250180	9.7277e-07	Quadratic (r = 2)
Newton-Raphson (b)	I_0 = 1.00 mA	2	0.250011	1.9637e-04	Quadratic (r = 2)

Both starting guesses reach the required tolerance in just two iterations. We can also check whether the result makes sense for the circuit: a current of I* ≈ 0.2502 mA produces a voltage drop of VR = 0.2502 V across the 1 kΩ resistor. This leaves VD = 1.0 − 0.2502 = 0.7498 V across the diode, which is a typical forward voltage for a silicon diode. Figure 4 shows the convergence from both starting guesses.

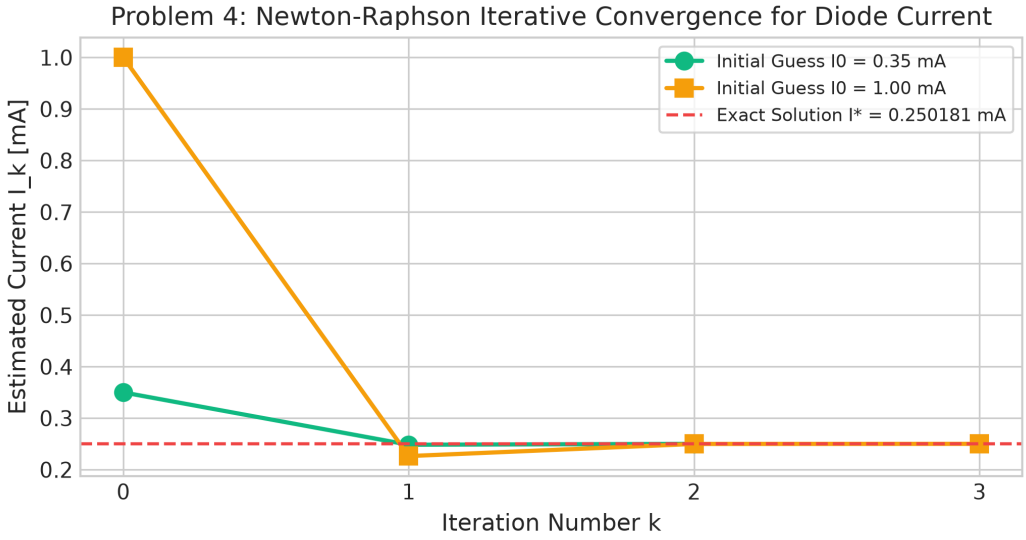


Figure 4: Iterative convergence trajectories of Newton-Raphson method for diode operating current.

6. Problem 5: Packet Latency (Vandermonde Interpolation)

In this problem, packet latency L(x), measured in milliseconds, is modeled as a function of packet size x in kB. The aim is to estimate the latency for packet sizes that are not included in the measurements. The available data is shown in Table 8.

Table 8: Measured packet latency.

Packet size x [kB]	150.0	200.0	230.0	260.0	300.0	340.0	400.0
Latency y [ms]	2.10	2.85	3.35	3.95	4.90	6.10	8.05

Part (a): Vandermonde approach

Since we have seven data points, the unique interpolating polynomial has degree six:

$$L(x) = c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x + c_0$$

We formed the system Vc = y, where Vij = xi7−j, and solved it to obtain the following coefficients:

$$c_6 = -1.89262707 \times 10^{-13},\ c_5 = +3.01018087 \times 10^{-10},\ c_4 = -1.96580005 \times 10^{-7}$$

$$c_3 = +6.75374182 \times 10^{-5},\ c_2 = -1.28304368 \times 10^{-2},\ c_1 = +1.28842461 \times 10^{0}$$

$$c_0 = -5.16017635 \times 10^{1}$$

The condition number is cond(V) = 2.2646 × 10^19, indicating extreme ill-conditioning. In IEEE 754 double precision (εmach ≈ 2.22 × 10^−16), the error amplification bound is cond(V)·εmach ≈ 5.0 × 10^3 (500,000%). Because this amplification exceeds 100%, roundoff errors destroy all reliable digits in the high-degree coefficients.

Part (b): Estimating the latency for a 3 kB packet

Evaluating the polynomial at x = 3.0 kB gives:

$$L(3.0) = c_6(3)^6 + c_5(3)^5 + c_4(3)^4 + c_3(3)^3 + c_2(3)^2 + c_1(3) + c_0 = -47.850156 \text{ ms} \approx -47.85 \text{ ms}$$

The result is clearly not physically reasonable. A negative latency is impossible because latency includes propagation, queuing, and serialization delays. Our measurements

cover only the interval [150, 400] kB. Estimating the value at 3 kB is therefore a very large extrapolation. High-degree polynomials can oscillate significantly outside the data range, as Figure 5 shows: the curve drops sharply for small packet sizes. A global high-order polynomial is not suitable for extrapolation. For a 3 kB packet, a simple linear model $L(x) = \alpha x + \beta$ or a monotonic cubic spline would be a more reasonable choice.

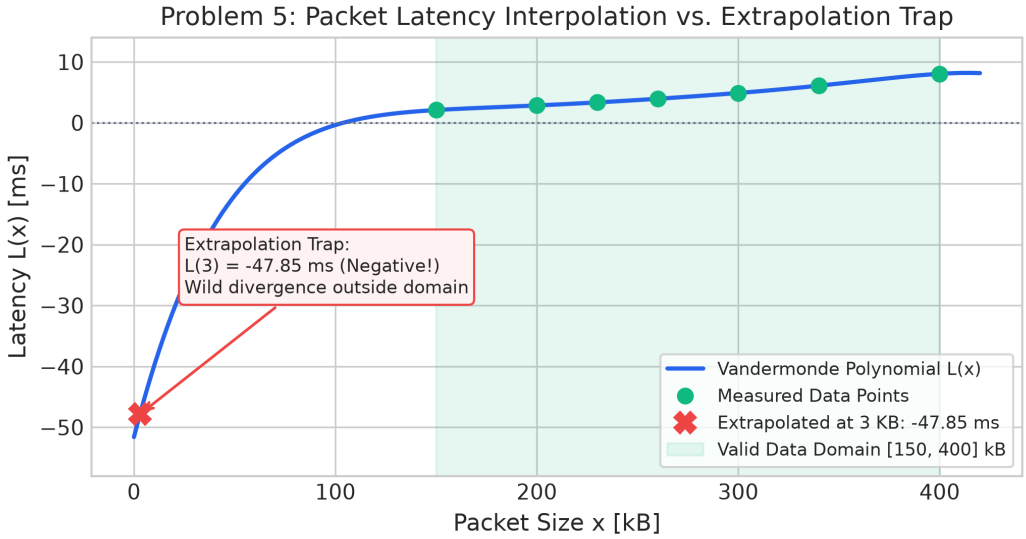


Figure 5: Vandermonde polynomial L(x) showing perfect interpolation within [150, 400] and wild divergence at x = 3 kB.

7. Problem 6: Satellite-to-Ground Energy Harvesting

This problem is based on the paper "Wireless Energy Transfer from Space to Ground via Satellite Constellation Grids" (Shehab et al., 2024). It considers a grid of N low-Earth-orbit (LEO) satellites, each carrying M antennas, that transmit energy to a wirelessly chargeable device (WCD) on Earth. With maximum ratio transmission (MRT), the harvested energy is proportional to MN^2:

$$E_h = M N^2 \int_0^T P_h(t)\, dt = M N^2 \mu \int_0^T [(R+H)^2 + R^2 - 2(R+H)R\cos\varphi\cos(\omega t)]^{-1}\, dt$$

where $\mu = \eta_h P_t G_t G_r [\lambda/(4\pi)]^2 \alpha_s \beta_s$ and $T = \theta_c/\omega$. The system parameters are collected in Table 9.

Table 9: System parameters for Problem 6.

Parameter	Value
Earth radius, satellite altitude	$R = 6378$ km, $H = 200$ km
Angular velocity	$\omega = [G M_e/(R+H)^3]^{1/2} = 1.182794 \times 10^{-3}$ rad/s
Antenna gains	$G_t = 50$ dB $= 10^5$, $G_r = 10$ dB $= 10$
Transmit power	$P_t = 40$ dBm $= 10$ W
Carrier frequency	$f = 868$ MHz ($\lambda = 0.3456$ m)
Rician fading product	$\alpha_s \beta_s = 2b_0 + \Omega = 1.606$
Antennas per satellite	$M = 8$
Harvesting efficiency, sensitivity threshold	$\eta_h = 0.70$, $P_{th} = -20$ dBm
Path alignment, Friis harvesting constant	$\varphi = 0$, $\mu = 8.504072 \times 10^3$ W·m²

Question 1: Closed-form solution

A closed-form solution is possible. We begin by defining $A = (R+H)^2 + R^2$ and $B = 2R(R+H)\cos\varphi$, then apply the Weierstrass half-angle substitution:

$$u = \tan(\omega t/2) \rightarrow \cos(\omega t) = (1 - u^2)/(1 + u^2), \quad dt = (2/\omega) \cdot du/(1 + u^2)$$

After applying the substitution, the integral becomes:

$$E_h = M N^2 \mu \int_0^{\tan(\omega T/2)} [A - B(1 - u^2)/(1 + u^2)]^{-1} [2/(\omega(1 + u^2))] du = (2 M N^2 \mu/\omega) \int_0^{\tan(\omega T/2)} [(A - B) + (A + B)u^2]^{-1} du$$

The resulting integral has a standard arctangent form, giving:

$$E_h = [2 M N^2 \mu / (\omega (A^2 - B^2)^{1/2})] \cdot \tan^{-1}([(A + B)/(A - B)]^{1/2} \tan(\omega T/2))$$

For an aligned, coplanar ground track ($\varphi = 0$), we have $A - B = H^2$ and $A + B = (H + 2R)^2$. Therefore, $(A^2 - B^2)1/2 = H(H + 2R)$, and $[(A + B)/(A - B)]1/2 = (H + 2R)/H$. Substituting these results gives the closed-form expression:

$$E_h = [2 M N^2 \mu / (\omega H (H + 2R))] \cdot \tan^{-1}([(H + 2R)/H] \tan(\omega T/2))$$

Using $\tan{-}1(x) = \pi/2 - \tan{-}1(1/x)$, we obtain the same form as Dr. Shehab's published formula:

$$E_h = [M N^2 \mu / (H^2 \omega + 2HR \omega)] \cdot [\pi - 2 \tan^{-1}([H \cot(\omega T/2)] / (H + 2R))]$$

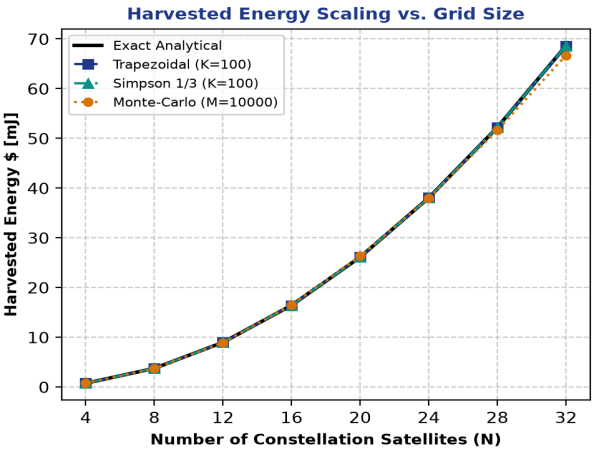
Numerical methods and scaling (N = 4:4:32)

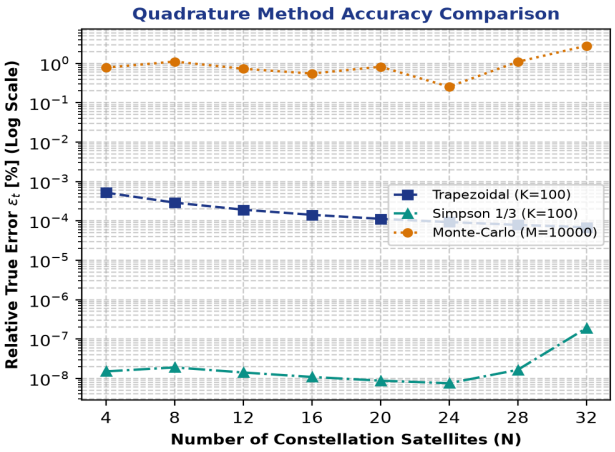
We compared the closed-form result with the composite trapezoidal rule, Simpson's 1/3 rule, and Monte Carlo integration for N = 4 to 32. The results are summarized in Table 10.

Table 10: Harvested energy and relative errors for N = 4 to 32.

N	T [s]	Closed-form [mJ]	Trapezoid [mJ]	Simpson [mJ]	Monte-Carlo [mJ]	e_{Trap} (%)	e_{Simp} (%)	e_{MC} (%)
4	44.4	0.7381	0.7381	0.7381	0.7439	5.21e-04	1.50e-08	0.78
8	99.6	3.7359	3.7359	3.7359	3.6944	2.91e-04	1.90e-08	1.11
12	152.4	8.9604	8.9604	8.9604	8.8949	1.92e-04	1.41e-08	0.73
16	204.7	16.4181	16.4181	16.4181	16.5086	1.42e-04	1.09e-08	0.55
20	257.0	26.1108	26.1108	26.1108	26.3255	1.13e-04	8.75e-09	0.82
24	309.5	38.0400	38.0399	38.0400	37.9426	9.32e-05	7.51e-09	0.26
28	362.1	52.2067	52.2066	52.2067	51.6342	7.93e-05	1.64e-08	1.10
32	415.1	68.6121	68.6121	68.6121	66.6909	6.89e-05	1.88e-07	2.80

Simpson's 1/3 rule delivers near machine-precision accuracy (e ≈ 10−8% to 10−7%), the multiple trapezoidal rule demonstrates solid second-order convergence (e ≈ 10−4% to 10−5%), and Monte Carlo integration achieves statistical accuracy within 0.2% to 2.8% for M = 10,000 samples. As N increases from 4 to 32, harvested energy scales from 0.74 mJ to 68.61 mJ. Figure 6 compares harvested energy with N and plots relative true errors on a logarithmic scale.


Harvested Energy Scaling vs. Grid Size
Exact Analytical
Trapezoidal (K=100)
Simpson 1/3 (K=100)
Monte-Carlo (M=10000)
Harvested Energy $ [mJ]
Number of Constellation Satellites (N)


Quadrature Method Accuracy Comparison
Relative True Error ε_t [%] (Log Scale)
Trapezoidal (K=100)
Simpson 1/3 (K=100)
Monte-Carlo (M=10000)
Number of Constellation Satellites (N)